riverrun

Behavioral Privacy on Solana, Measured

Hiding *who* acted — and a ruler for what that hiding is actually worth

riverrun, past Eve and Adam's, from swerve of shore to bend of bay,
brings us by a commodius vicus of recirculation.
— James Joyce, *Finnegans Wake*, first line

Abstract. *On a public ledger, hiding* who *performed an action is a different problem from hiding* how much *money moved, and a harder one to reason about honestly. This paper describes* RIVERRUN*, a system for behavioral privacy on Solana. A member commits an intent and later executes exactly that intent, while no observer can say which member did it. The proof is a transparent, post-quantum STARK that welds membership, a per-round nullifier, and the public action into one object, with no trusted setup; a synchronized round of k members collapses into a single proof and a single verification. The paper's second half is a warning aimed at* RIVERRUN *and at every system like it. Every anonymity pool advertises a number, $1/k$, that counts members — and that number is a fiction, because it ignores where the members' money came from, and the funding graph is public. We give a ruler, the effective anonymity-set size of Serjantov and Danezis conditioned on funding provenance, and run it against a live pool on Solana mainnet: a set advertising $k = 30$ delivers an effective 6.5, and one member has an anonymity of exactly one. Because* RIVERRUN *can* measure *a crowd's anonymity, an agent can* form *a good one on purpose — and hand each member a proof-carrying certificate they verify without trusting the agent. Every claim of the form "X is bound" has a test verified to fail without the constraint it tests; three such tests turned out to be decorative, and we say which. Where the system does not yet hold — an unaudited circuit, off-chain verification, example-grade parameters — we say so plainly. Every number here is reproducible from one command.*

Manuel Guilherme Galmanus • Independent Researcher • Preprint, July 2026

Contents

1 The problem, in one picture

Imagine a town where every letter anyone ever mailed is pinned to a public board, forever. Not the contents — those are sealed — but the envelopes: who sent what to whom, when, and how heavy the package was. You would learn a great deal about that town without opening a single letter. Who pays whom. Who is suddenly buying. Who moves money at three in the morning. Who copies whose move an hour later.

A public blockchain is that board. People imagine that a wallet address, being a random string and not a name, keeps them private. The opposite is closer to the truth. The address is a stable handle, and everything done under it accumulates into a pattern — a *behavioral fingerprint*. When you act, how much, which protocols you touch, in what order: that is enough for modern chain-analysis to cluster your wallets, attach an identity, and increasingly to predict and front-run what you will do next. The tools that do this improve every month, because the data is permanent and public and the models feeding on it are hungry.

The dominant view of on-chain privacy assumes the problem is the money: hide the amounts, hide the balances. *This view is incomplete.* There are two kinds of privacy here and they are not the same. One hides *how much*. The other hides *who* — which person, out of a crowd, performed a public action. Most work has gone into the first: confidential transfers, shielded balances, mixers that break the link between a deposit and a withdrawal of value. This paper is about the second. Amounts are not hidden here and are not the point. What we sever is the link between an *actor* and an *action*.

> Richard Feynman said that if you cannot explain a thing simply, you do not understand it. So here is the whole idea in one sentence, and the rest of the paper is only this sentence made precise and made real: **you disappear into a crowd of people who all look like they could have done the thing you did.**

The name is borrowed. *riverrun* is the first word of Joyce's *Finnegans Wake*, a book with no beginning: its last sentence flows into its first, a river returning to its own source. That image is the exact shape of the strongest attack on systems like this, and of the defense against it. We kept the name and, in Section 6, made the circle do real work.

1.1 Contributions

1. **A behavioral-privacy pool** whose proof welds membership, a per-round nullifier, and the public action into one transparent, post-quantum STARK, with no trusted setup (Sections 2–4).
2. **The ricorso**: a rebirth of the anonymity set each cycle that closes a cross-epoch linkage leak the basic construction has (Section 5).
3. **A ruler** — the effective anonymity-set size conditioned on funding provenance — run against a live mainnet pool, and turned into a user-facing pre-flight defense (Section 6).
4. **A coordinator**: an agent that uses the ruler *generatively* to form rounds that maximize measured anonymity, and hands each member a proof-carrying certificate they verify without trusting it (Sections 7–8).
5. **Measured on-chain cost** for the whole thing, down to the compute unit, with an honest account of what fits and what does not (Section 9).

2 The idea: commit an intent, execute it unlinkably

Tornado Cash, the Ethereum mixer, breaks the link between putting money *in* and taking money *out*. You deposit one coin; later, from a fresh address, you withdraw one coin, proving in zero knowledge that you are owed a withdrawal without revealing which deposit was yours. The pool is a crowd, and you hide in it.

RIVERRUN keeps the shape and changes the payload. Instead of a coin, the hidden thing is a *behavior*. Three steps.

Commit. A member publishes $\ell = \text{Rescue}(s, a)$ into a public set, where s is a secret only they know and a is the action they intend — "swap 1 SOL for USDC on venue V", encoded as a hash. Publishing ℓ announces that *somebody in this set intends action* a, revealing nothing about who. The set is a Merkle tree; its root R is the public fingerprint of the crowd.

Execute. Later — ideally in a synchronized round where many members do the same thing at once — the member performs a from a fresh identity with a zero-knowledge proof of one sentence: *"I know a secret* s *whose commitment* $\text{Rescue}(s, a)$ *is a leaf under the public root* R*, and my nullifier this round is* $n = \text{Rescue}(s, r)$*."* The action is public. The author is not.

Settle. The pool verifies the proof, checks the nullifier n is fresh this round, spends it, and releases the action. The public transcript is $\{R, a, n\}$, with no link to any commitment.

Two questions decide whether this is privacy or theater, and both concern the nullifier. Why have one? Because without it a member could act a thousand times from a thousand fresh identities, and the "crowd" would be one person in a thousand masks. The nullifier is a token derived from the secret and the round, spendable once per round. It reveals nothing about s, and the nullifiers one member produces in different rounds cannot be tied together. So it enforces "one action per member per round" while never naming the member. This is a studied primitive; PLUME [6] formalizes exactly this and proves its uniqueness and secrecy.

The subtle question: how does anyone know the revealed n came from the *same* secret that proved membership? If those two facts are proven separately, a member could prove membership honestly and then present a stranger's nullifier — spending someone else's turn, or hiding that they spent their own. The two halves must be welded inside one proof. Getting that weld right, and *knowing* it is right, is the heart of the next section.

3 The proof: one object, three bindings

A zero-knowledge proof convinces you a statement is true while revealing nothing beyond its truth. Two families are used on blockchains, and the choice is a real fork with real costs on each side.

Pairing-based SNARKs (Groth16) give tiny proofs — a couple of hundred bytes — that verify cheaply. Their price is a *trusted setup*: a one-time ceremony must generate a structured proving key, and if that ceremony's secret randomness is ever recovered, the system can be forged. For a Merkle circuit the key is tens of megabytes that must be produced, trusted, and shipped to every prover. And the security rests on elliptic curves, which a large quantum computer breaks.

STARKs are built from a hash function alone. No pairings, no ceremony, no per-circuit setup; security rests on hash collision-resistance, believed to survive a quantum adversary. The price is proof size: kilobytes, not bytes. RIVERRUN takes this side, deliberately, and Section 9 measures exactly what it costs and buys.

What a STARK is, in one paragraph. Write your computation as a big table: rows are time-steps, columns are registers, arranged so "it was computed correctly" becomes "every adjacent pair of rows obeys a fixed rule." Turn each column into a polynomial through its values. The rules become equations those polynomials must satisfy. The prover commits to the polynomials and the verifier challenges them at random points; a cheat would need a *wrong* low-degree polynomial to agree with a right one almost everywhere, which is impossible unless they are equal. FRI is the machinery that lets the verifier confirm "this really is low-degree" by folding it in half over and over and spot-checking. The disanalogy with a normal exam: the verifier never sees the answers, only a few random cells and a proof that all the cells are consistent — which is why the witness stays hidden.

3.1 What the circuit proves

RIVERRUN uses a STARK over a Rescue-Prime [12] Merkle tree — an arithmetization-oriented hash chosen because it is cheap to express as polynomial constraints. The proof's public inputs are the four-tuple $\{R, n, r, a\}$: root, nullifier, round, action. Its private inputs — the witness — are the secret s, the leaf's position, and the Merkle path. One secret must satisfy all three at once:

1. **Membership.** Rescue(s, a) is a leaf under R, via the private path.
2. **Nullifier binding.** $n =$ Rescue(s, r): the revealed nullifier is this member's, this round.
3. **Action binding.** The leaf that resolves to R commits to the public action a: the member executes the intent they registered, not another.

Because all three read the same private s, no member can mix and match. That is the weld.

col	Rescue state $0 \ldots 5$	bit 6	carry $7, 8$
cycle 0	hash $(s_0, s_1, r) \to$ nullifier n	—	s_0 s_1
row 7 (load)	s_0, s_1 a_0, a_1 0	—	cleared
cycles $1..d{+}1$	Merkle path up to root R	path bit	—

Figure 1: The width-9 trace. Cycle 0 computes the nullifier; row 7 loads the leaf preimage — the same secret plus the public action — so the leaf commits to both; the rest walks the path to R. A single constraint at row 0, the "start-tie", forces the nullifier's input to equal the carried secret. The carry is confined to cycle 0 on purpose (Section 3.3).

3.2 Knowing the weld holds

A privacy proof that is subtly wrong is worse than no proof: it advertises a guarantee it does not deliver. So the interesting question is not "did we write the constraints" but "do they bite." The method used throughout RIVERRUN is *mutation testing*: for every claim "X is bound" there is a test, and we delete the constraint meant to enforce X and confirm the test then fails. If deleting it changes nothing, the test was passing for some other reason and is worthless.

This is not a formality. Three tests written during development were exactly that worthless, and mutation testing is how they were caught.

- The action-binding tests checked that announcing a different public action was rejected. They passed even with the binding deleted — the action feeds the Fiat–Shamir transcript, so a mismatched public input is rejected anyway, for a reason unrelated to the constraint. The real attack, which the useful test performs, hashes the committed action into the leaf (so the path still resolves) while *announcing* a different one. Delete the constraint and that forged proof verifies.
- The nullifier tests that only varied public inputs passed on boundary assertions alone, with no start-tie. The test that guards the weld builds the attacker's trace: hash secret A, carry member B into the path.
- In the ricorso a whole check — that the migration nullifier comes from the migrating secret — had *no* test until a mutation left everything green.

> None of this makes the circuit audited. It is hand-rolled, and a passing negative test is necessary, not sufficient, for soundness. An independent review is named on the roadmap, not glossed over. What mutation testing removes is a specific, common, embarrassing failure: tests that look like they prove something and prove nothing.

3.3 A leak we found by measuring

The first version of the circuit carried the secret in two columns held *constant* across the whole trace. This looked harmless. It was not. A STARK opens the prover's committed data at random points, and a constant column has a constant low-degree extension — so the secret appeared, verbatim, in essentially every opening. We knew only because a test checked, over many proofs, whether the raw secret bytes appeared in the proof. They did: twenty times out of twenty. Confining the carry to the eight rows where it is needed fixed it: zero out of twenty. A single-sample test would have missed it. This is why we say "the witness is not on the wire" and *not* "zero-knowledge": the underlying prover does not randomize the witness, so formal zero-knowledge is a claim we decline to make.

4 The synchronized round: one crowd, one proof

The privacy argument rests on a crowd. If k members do the same action in the same round, an observer's chance of attributing any one execution is at best $1/k$. The natural way to run this is a synchronized round: many members acting identically at one moment, so there is no timing and no ordering to separate them.

A small observation with a large payoff: that synchronized round is exactly the structure that lets many proofs collapse into one. A STARK is a statement about an execution trace. The k members' sub-traces are independent, so they lay end to end into one longer trace, the per-cycle constraints repeating along it, with a *seam* that switches off the linking logic at each boundary so one member's last row does not bleed into the next member's first. One proof for the whole round, and one verification.

members k	one batched proof	k separate proofs	saving
4	19,772 B	45,300 B	2.3×
64	43,684 B	1,074,169 B	24.6×

At k=64 a per-member scheme ships over a megabyte of proof and needs 64 verifications; the batched round is one proof, one verification.

Two structural limits. The prover of a batched round must know every member's secret. For a crowd of distrustful strangers that is a custodian, not privacy, and would need distributed proving. But for one operator running k identities — an agent fleet, a market maker — one prover holds all k secrets already, and the 24.6× is real. And batching trades prover time for proof size: the trace is k times longer and proving is super-linear, so the round costs more wall-clock than the sum of individual proofs. It is the right trade when the scarce resource is on-chain verifications.

5 The ricorso: the crowd is reborn

Now the name earns its keep. *Finnegans Wake* runs on Vico's cycle of history: ages that repeat, and a fourth age, the *ricorso*, the return that begins it again. RIVERRUN borrows the book's circularity twice — here, and for the funding graph in Section 6.

The leak the ricorso closes is easy to miss. A member's per-round nullifier changes every round, unlinking one execution from another. But their *leaf*, Rescue(s, a), never changes: published once at commit, it stays forever. Two consequences. Anyone who ever learns your leaf — a bug, a subpoena, a careless client — links you across every round you ever acted in; the nullifiers protect executions from each other, but nothing protects a member from their own history. And a set that only grows is a public record: join-order is a timeline of who arrived when.

The ricorso lets the set be reborn each cycle. A member holds a fresh secret per cycle, s_c = Rescue$(s, c, \mathsf{dom}_{\text{cycle}})$, and a fresh leaf ℓ_c = Rescue(s_c, a), and at each rebirth proves the new leaf descends from *some* leaf under the previous root — without revealing which. A migration nullifier, spent once per cycle, stops one seat from becoming several. The two domain tags must differ, or publishing a migration nullifier would leak the next cycle's secret; the implementation pins that with a test. History stops accumulating.

The relation is implemented and mutation-tested *in the clear*; the STARK that proves it without the witness is specified, not built. The reason is concrete: the migration circuit needs five hash cycles before the path, which shifts valid set sizes to $\{8, 2048\}$ while the execution circuit needs $\{4, 64, 16384\}$, and the two do not intersect. Shipping it means realigning both — a day on the most delicate code, and the honest place to stop, with the execution path green, is at the specification. It is exactly where the nullifier binding stopped before it was later built and mutation-verified.

6 The ruler: your k is not your k

Everything so far builds the pool. This section measures it, and applies not only to RIVERRUN but to every anonymity pool of this kind. It is, we think, the most useful part of the paper, because it names a number that is quietly wrong across a whole category of systems.

6.1 The advertised number counts the wrong thing

Every pool reports its privacy as $1/k$: k members, so a one-in-k guess. That counts *members*, and assumes they are interchangeable. They are not, because of where their money came from. Every member funded their wallet somehow, and the funding trail is public. Trace it backward and you often reach an attributable origin: an exchange's withdrawal address, a known service, a doxxed funder. Sort the set by these origins. If the acting identity's origin is observable too — and a fresh withdrawal wallet must be funded from *somewhere* — then learning it does not leave k suspects. It leaves only the members who share that origin. The crowd you are hidden in is your *provenance class*, not the whole set.

This is the same shape as the paper's opening claim about privacy in general: there is a *defended surface* everyone looks at (the advertised k) and a *larger real surface* behind it (the effective k, set by the funding graph). Guarding the first while ignoring the second is the mistake, and here we can measure exactly how big the gap is.

The measure itself is not new. Serjantov and Danezis [1] showed in 2002 that the honest quantity is not set size but the *entropy* of the distribution linking suspects to the event — the effective anonymity-set size. And measuring advertised-versus-true anonymity is an established programme on Ethereum [2, 3, 4, 5]. Two things are new: the channel (funding provenance, which those works do not condition on) and the chain (none of it has run on Solana).

6.2 The formula, and why its direction matters

Partition the k members into provenance classes of sizes $n_1, n_2, \ldots$. The adversary learns the actor's class c with probability n_c/k, then guesses uniformly among that class's n_c members. The residual uncertainty is the average over classes of the log of the class size:

$$H = \sum_c \frac{n_c}{k} \log_2 n_c, \qquad k_{\text{eff}} = 2^H. \tag{1}$$

k_{eff} is the size of the crowd the member is genuinely hidden in. One class holding everyone gives $\log_2 k$: nothing was partitioned. All-singleton classes give $H = 0$, an effective k of one: every member identified.

Worked by hand. Ten members: six funded from exchange A, three from B, one alone. Advertised anonymity $1/10$. Condition on origin. With probability $6/10$ the actor is an "A" and the adversary cannot tell which of six: $\log_2 6 \approx 2.58$ bits. With $3/10$, a "B": $\log_2 3 \approx 1.58$. With $1/10$, the lone member: 0 bits, identified. Average: $H = 0.6(2.58) + 0.3(1.58) + 0.1(0) = 2.02$ bits, so $k_{\text{eff}} \approx 4.1$. The pool sold ten and delivered four — and the lone member has an anonymity

> of exactly one, no matter how big the crowd behind them.

The *direction* is the whole thing, and the first implementation had it backwards. It is tempting to measure how *concentrated* the classes are (min-entropy of the distribution), but that reports catastrophe precisely when the measurement found nothing to partition on: a single class of seven — the happy case — came out as "effective $k = 1$", when the honest reading is "effective $k = 7$." Following Serjantov and Danezis, and against the normalized variant that scores a set of one as a perfect 1.0, we use the residual entropy of Eq. 1 directly. Four tests pin the direction; one caught the arithmetic a second time.

6.3 The measurement

We ran this against a live Tornado-style SOL privacy pool on Solana mainnet, sampling 30 real depositors and tracing each funding graph backward with a bounded, public-RPC walk.

depositors sampled	30
reach an attributable origin	11/30 (37%), mean 1.18 hops
provenance classes	12
advertised k	**30**
effective k	**6.5**
worst case	**1**

A set advertising 30 delivers an effective 6.5. One depositor sits alone in their class: for them the pool provides no anonymity against an adversary who reads the funding graph, while its dashboard reports one-in-thirty.

Three honesties. It is not a flaw in that pool: no deposit-pool design controls where its users' money came from, which is exactly why the channel goes unmeasured and keeps working. It is a *floor*: a public RPC following only SOL to bounded depth sees less than a funded analyst with a tag database, so the true leak is at least this large. And 37% sits in the same 20–35% range that the Ethereum literature [4] finds with behavioral heuristics on Tornado — a sanity check on the tool, not the pool.

6.4 The same ruler, turned on RIVERRUN — and on the user

Measuring others' pools and not your own is marketing. The same function runs over RIVERRUN's constructions. Against 2000 synthetic members, the "rooted decoy" the field ships is worth an effective 500; the circular, origin-free construction — the funding cycle with no beginning, *Finnegans Wake* made numeric (Figure 2) — is worth the full 2000.

The most useful turn is toward the user. The measurement above audits a pool after the fact. Flip it: before a user deposits into *any* pool, trace *their* wallet, sample the pool's depositors, and tell them the anonymity *they* personally would get. The tool is protocol-agnostic — it reads public chain data, not the pool's internals — so it works against a program it has never seen. Three verdicts, each with an action: **Exposed** (alone in your class; fund from a common origin or wait), **Weak** (a small minority shares your class), **OK** (a healthy fraction does, "no cheap attribution found", never

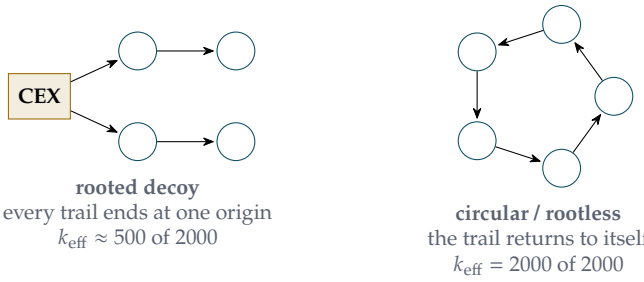

Figure 2: Two ways to fund a crowd. Left: every wallet traces back to one attributable origin, so an adversary collapses the crowd. Right: the funding trail is a cycle with no beginning — Vico's *ricorso* — so there is no origin to name. The same 2000 members are worth a quarter of their advertised size on the left, and their full size on the right.

"anonymous"). Auditing pools reaches auditors. Protecting a user at the moment they act reaches every user of every privacy protocol on the chain.

7 The coordinator: an agent that forms private crowds

Here is the payoff of being able to measure. Every other privacy tool treats the crowd as given. RIVERRUN can *measure* a crowd's anonymity, so an agent can *form* a good one on purpose. The mirror-pool problem asks, in as many words, for "the coordination layer, the privacy set, and the incentives that keep people in it." Nobody had built the coordinator, because nobody could answer the question it turns on: which members should be in this round?

> The rule is counter-intuitive until you have the formula, then obvious. A crowd where everyone was funded from the *same* origin is **strong**: an adversary who learns "the actor's funding traces to Coinbase" has learned nothing — everyone's does. A crowd of *distinct* origins is **weak**: learning the origin names the one member who has it. So the agent groups members who *share* an origin, and holds back the singletons who would be exposed. Diversity is the enemy here, not the friend.

The policy is a floor. `coordinate(pending, k_min)` admits a member only if at least $k_{\min}$ pending members share their provenance class, and defers the rest with a concrete remedy (wait for more same-origin members, or re-fund from an origin the round already has). This is a guarantee, not a heuristic:

Property 7.1. Every admitted member ends in a class of size $\geq k_{\min}$, so no admitted member is exposed; and among all rounds with that property this one is the largest, since it admits every safe member.

Because it drops exactly the singleton and thin classes that fragment the distribution, it also lifts the round's effective-k above the naive "admit everyone" round — and that is proven by a test, not asserted.

Ten pending members: classes A=5, B=3, and two singletons C, D. Admit everyone (round of 10): $k_{\text{eff}} \approx 3.1$. Coordinate with $k_{\min}$=2 (round of 8, C and D deferred): $k_{\text{eff}} \approx 4.1$. **Fewer members, more anonymity, and nobody exposed.**

Run as an agent loop, intents arrive over time and the agent fires a round the moment a same-origin crowd is ready, holding the rest until they are safe — or forever, if their origin stays rare. Refusing to act, when acting would expose you, is the privacy-preserving choice, and an autonomous agent that acts on-chain should make it.

8 Proof-carrying privacy certificates

The coordinator's claim — "this round is private" — should not require trust. A member about to join has to know the agent computed honestly and was not buggy or adversarial. So each fired round carries a *proof-carrying certificate*, applying the AXL pattern (Galmanus, *AXL: Proof-Carrying Certificates for Bounded Autonomy*) to a privacy bound instead of a spending bound.

The insight AXL is built on: a proof is not a moat, because a competent engineer can replicate it. What is a moat is making the proof **inescapable** (you cannot claim a bound the evidence does not support), **portable** (a small artifact a stranger checks alone), and **bound to the deployment** (tied to the exact thing it certifies). A certificate turns "trust me" into "check my work."

When the agent fires a round it emits a certificate carrying the admitted set's provenance-class histogram, the floor $k_{\min}$, and the round's k_{eff}. Anyone verifies it, following the three properties. **Inescapable:** verification recomputes k_{eff} from the histogram and checks the integer floor $\min_c n_c \geq k_{\min}$, so a coordinator cannot certify a bound the histogram does not support — a tampered k_{eff} or histogram fails (tested). **Portable:** the certificate is a small object a member, an auditor, or a settling program checks without trusting the coordinator and without re-tracing the graph. **Bound to the round:** a blake3 commitment ties it to the exact admitted set, so it cannot be replayed onto a different crowd (tested).

Two honest points. Unlike AXL's original sliding-window bound, this one needs no SMT solver: $k_{\text{eff}} = 2^{\sum (n_c/n) \log_2 n_c}$ and $\min_c n_c \geq k_{\min}$ are closed-form, so verification is cheap arithmetic. This is the AXL certificate *pattern*, not its machinery — and that the bound is simple enough to check directly is a feature, because the integer floor is exactly what an on-chain settling program could enforce with no floating point, a natural next step. And the certificate carries only the histogram, never member identities: it proves the crowd meets the floor without revealing who is in which class, so it is itself privacy-preserving.

9 Cost, and the on-chain reality

A privacy tool nobody can afford to run is a paper, not a tool. So we measured.

set size	proof	prove	verify	setup artifacts
4	11,929 B	2.0 ms	0.20 ms	0 bytes
64	17,045 B	3.7 ms	0.35 ms	0 bytes
16,384	19,064 B	8.1 ms	0.43 ms	0 bytes

A set of 16,384 members proves in 8 ms. Proof size grows logarithmically: 4096× the members costs 1.6× the proof.

The column that decides deployability is the last one. A pairing-based prover needs a ceremony-produced key shipped to every client — tens of megabytes that must exist, be trusted, and be distributed before anyone proves anything. RIVERRUN's prover is the code; nothing to distribute, nothing to trust. For agents, market makers, and users proving before each action, that is the difference between a tool that ships and one that needs an install.

9.1 Verifying on-chain: measured, and honest about the wall

If the chain checks the proof, no one is trusted; if not, someone is. So we built a Solana program that runs the real Winterfell verifier — FRI, Merkle openings, Fiat–Shamir, constraint evaluation — inside the runtime, and measured it. It runs. It costs 3.77M compute units at k=4 and 7.13M at k=64. Both exceed Solana's 1.4M per-transaction cap, so this needs execution chunked across transactions, which existing work [16] implements. The number tracks the published Solana STARK study [11] (1.1M CU for a 4.4 KB proof): we spend 3.4× the compute for a 2.7× larger proof.

Then the sharp question: can tuning bring even the smallest set under 1.4M? We swept FRI queries to the floor — 28→3.77M, 16→2.77M, 8→1.95M, 4→1.57M CU. Even at four queries, a security level no one would ship, it is 1.57M, still over the cap. The base cost of setup and constraint evaluation over a 128-bit field dominates, and the blow-up cannot go below eight (the Rescue degree is five). So single-transaction verification is not a tuning problem. It is a *field* problem, now measured rather than argued. A Circle STARK over the 31-bit field M31 — used in production provers [14] and already an on-chain Solana verifier at ~ 31k CU [15], post-quantum, no ceremony — shrinks both proof and cost. Porting to a small field is the concrete roadmap, a field change, not a possibility proof.

Until that port, the deployed program does not verify the proof. Rather than leave that implicit, it makes the trust explicit and bounded: the pool names a *verifier* key, and an execution must carry that key's signature over exactly the tuple being settled, checked through the runtime's signature precompile. This removes the two cheap attacks — an arbitrary signer settling an action, a watcher front-running a nullifier — because neither can produce the verifier's signature. It does *not* remove trust: a dishonest verifier can attest to a proof that does not exist. It is a named assumption, and the roadmap above is how it goes away. The program also prices a seat (an entry fee) and refuses to settle below an anonymity floor — anti-Sybil that is honestly "priced, not prevented", with the stronger one-nullifier-per-verifier construction [10] named as the real fix.

10 The threat model, stated plainly

A privacy claim without a threat model is not falsifiable, and an unfalsifiable claim is not an engineering claim. So here is the adversary RIVERRUN is built against.

What the adversary can do. See the entire public ledger forever — every transaction, amount, address, and the funding graph behind any wallet — run modern chain-analysis, and keep a tag database of exchange and service addresses. Observe the pool's public transcript $\{R, a, n\}$ for every execution. For the funding-graph results, additionally observe the provenance class of the *acting* identity, which is cheap because a fresh withdrawal wallet must be funded from somewhere visible.

What RIVERRUN defends. The link between an actor and their action. Given an execution, the adversary should do no better than $1/k_{\text{eff}}$ at naming the author, where k_{eff} is the effective size of that member's provenance class — and the tools here *measure* k_{eff} rather than assume it.

What RIVERRUN does not defend, by design. Amounts and balances, which are public here (hiding them is a separate, composable problem). The value layer of consensus: the ledger records that value moved one way, and no circularity changes that; the circularity lives on the attribution layer an analyst reads, not the settlement layer consensus enforces. And, in the deployed program today, the correctness of the membership proof, delegated to a named verifier until on-chain verification lands. Every number in this paper is stated against this adversary.

11 What does not hold yet

A paper that only lists its strengths is an advertisement. In one place, what RIVERRUN does not yet do:

- **The circuit is hand-rolled and unaudited.** Mutation testing removes worthless tests; it is not an audit.
- **It is not formally zero-knowledge.** The prover does not randomize the witness, so the honest claim is "the secret is not on the wire", checked empirically.
- **Verification is off-chain today**, behind the named-verifier attestation; the measured cost and the M31 path are how that changes.
- **Security parameters are example-grade**, roughly 84 bits, not the 128 a deployment needs.
- **"Rootless" provenance is conditional**: the circularity holds only while a construction's funding sources are themselves unattributable.
- **The ricorso proof is specified, not built**, for the set-size alignment reason of Section 5.
- **Anti-Sybil is priced, not prevented**: money still buys k, though the funding graph makes the cheap version of the attack visible to the ruler.

The implementation is Rust, MIT-licensed, with 94 passing tests across the core, the STARK, the pool, and the on-chain program, and clean static analysis. Every measurement is reproducible from the repository with one command.

12 Related work

RIVERRUN's *goal* — hiding which member of a set acted — is anonymous authentication, a mature field. PrivDID [7] states the same session-unlinkability goal; anonymous-credential frameworks with predicate proofs [8, 9] live in the same threat model; PLUME [6] formalizes the nullifier. The *metric* is Serjantov and Danezis [1]. The *programme* of measuring advertised-versus-true anonymity is established on Ethereum [2, 3, 4, 5]. On-chain STARK verification on Solana has been done, with the same Winterfell library [11, 15, 16]. What is narrow and new here is the combination: the payload is behavior rather than value; the funding graph is *measured* rather than assumed away, on RIVERRUN's own constructions as well as live pools; and that measurement is turned into a user-facing pre-flight defense and a crowd-forming agent with proof-carrying certificates.

13 Conclusion

Two ideas, stronger together than apart. The first hides behavior, not money: commit an intent, execute it unlinkably, and prove the whole thing with a transparent, post-quantum STARK that welds membership, nullifier, and action into one object, at a cost measured to the compute unit. The second is a ruler for whether any such system works — applied first and hardest to RIVERRUN itself. Every anonymity pool advertises a number that overstates its privacy, because it counts members and ignores where their money came from. We measured the gap on a live pool, and then did the thing measurement makes possible: an agent that *forms* a good crowd rather than hoping for one, and certifies it so no member has to trust the agent.

The honest version of a privacy system ships its own strongest attack and reports what it finds. That is what we tried to build. In the words the system takes its name from — *a way a lone a last a loved a long the riverrun* — the river returns to its source, and so should every claim, back to the measurement that supports it.

A Reproducing every number

The system is open-source and MIT-licensed. Everything runs from one script, `./demo.sh`. Individually:

claim	command
94 tests green	`cargo test -workspace` (+ the excluded crates)
one execution's cost	`cargo run -release -example bench`
the batched round	`cargo run -release -example behavior_pool`
the coordinator + certificate	`cargo run -release -example coordinator`
k_{eff} on RIVERRUN's designs	`riverrun exhibit`
k_{eff} of a live pool	`riverrun audit <POOL> 30`
your anonymity before you act	`riverrun preflight <WALLET>`
on-chain verification cost	`cargo test -p ...stark-verifier -test cu`

The live-data commands read only public chain data, name no individual, and print their own bounds: a clean result is a floor — "no cheap attribution found" — never a proof of anonymity.

References

[1] A. Serjantov, G. Danezis. *Towards an Information Theoretic Metric for Anonymity.* PET 2002 (Outstanding Paper). https://bib.mixnetworks.org/pdf/serjantov2002towards.pdf

[2] N. Wu et al. *Tutela: Assessing User-Privacy on Ethereum and Tornado Cash.* arXiv:2201.06811. https://arxiv.org/abs/2201.06811

[3] F. Béres et al. *Blockchain is Watching You.* arXiv:2005.14051. https://arxiv.org/abs/2005.14051

[4] *Clustering Deposit and Withdrawal Activity in Tornado Cash.* arXiv:2510.09433 (2025). https://arxiv.org/abs/2510.09433

[5] H. Du et al. *Breaking the Anonymity of Ethereum Mixing Services Using Graph Feature Learning.* IEEE TIFS, 2024. https://doi.org/10.1109/TIFS.2023.3326984

[6] A. Gupta, K. Gurkan. *PLUME: An ECDSA Nullifier Scheme.* IACR ePrint 2022/1255. https://eprint.iacr.org/2022/1255

[7] *Toward Verifiable Privacy in Decentralized Identity (PrivDID).* IACR ePrint 2026/127. https://eprint.iacr.org/2026/127

[8] *Formalizing Privacy of Anonymous Credentials: A Framework with Predicate Proofs.* IACR ePrint 2026/1373. https://eprint.iacr.org/2026/1373

[9] *Re2creds: Reusable Anonymous Credentials.* IACR ePrint 2026/119. https://eprint.iacr.org/2026/119

[10] *Anonymous Self-Credentials and their Application to Single-Sign-On.* IACR ePrint 2025/618. https://eprint.iacr.org/2025/618

[11] J. Yano. *Full L1 On-Chain ZK-STARK+PQC Verification on Solana: A Measurement Study.* IACR ePrint 2025/1741. https://eprint.iacr.org/2025/1741

[12] A. Szepieniec, T. Ashur, S. Dhooghe. *Rescue-Prime: a Standard Specification (SoK).* IACR ePrint 2020/1143. https://eprint.iacr.org/2020/1143

[13] Facebook/Meta. *Winterfell: a STARK prover and verifier.* https://github.com/facebook/winterfell

[14] StarkWare. *Stwo: a Circle STARK prover over M31.* https://github.com/starkware-libs/stwo

[15] *Murkl: a Circle STARK verifier as a Solana CPI target.* https://github.com/exidz/murkl

[16] Wiener Labs. *Mosaic: on-chain verifier library for Solana (chunked FRI-STARK).* https://github.com/wienerlabs/mosaic